DESIGN QUESTIONS

Design an E-commerce System

Microservices, Inventory Management & Order Orchestration

~ 35 minute read

© Study Guide — For personal use only

What's Inside

01 · Problem Overview
02 · Clarifying Questions
03 · Capacity Estimation
04 · System Architecture
05 · Core Design Decisions
06 · Data Models
07 · Order Placement Pipeline
08 · Inventory & Concurrency
09 · Search Architecture
10 · Recommendations
11 · Failure Modes
12 · Interview Playbook

01 Problem Overview

Building Amazon-Scale E-commerce

Designing an e-commerce platform requires balancing availability, consistency, and scale. Amazon handles 500M products, 1B users, and processes 100K+ orders per day at peak.

The Scale Problem

- **Users:** 1 billion registered; 300M monthly active
- **Catalog:** 500M products across all categories (electronics, clothing, books, groceries, etc.)
- **Peak traffic:** 100K+ orders/day; 10K concurrent users during sales/holidays
- **Search QPS:** 500K queries/second globally
- **Read-heavy:** 99% reads (browse, search, reviews) vs. 1% writes (orders, updates)

Key Challenges

- **Inventory consistency:** Prevent overselling; real-time reservation conflicts across multiple regions
- **Order atomicity:** Payment → inventory deduction → fulfillment must be transactional
- **Search relevance:** 500M products; millisecond response; personalization based on user history
- **Global distribution:** Regional warehouses, VAT compliance, currency conversions
- **Recommendations:** Personalized homepage; 'Customers also bought' at scale

02 Clarifying Questions

Scope & Assumptions

Question	Answer / Assumption
Single product or multi-vendor marketplace?	Single vendor (Amazon.com); seller accounts optional (Phase 2)
Global or single-region?	Global (multi-region: US, EU, APAC); local currency & tax
Inventory real-time or eventual consistency ok?	Real-time for high-velocity SKUs; eventual ok for slow movers
Search: full-text only or faceted?	Full-text + facets (price, rating, seller, availability)
Return / cancellation handling?	Out of scope; assume successful orders only
Payment processing (in-house or 3rd party)?	3rd party (Stripe, PayPal); async webhook confirmation
Seller onboarding (FBA, FBM)?	FBA only (Fulfillment by Amazon); logistics out of scope

© Study Guide — For personal use only

03 Capacity Estimation

Numbers to Anchor Your Design

Product Catalog

- **500M products × 2KB average metadata:** ~1TB in product catalog DB
- **Storage for images:** 5 images/product × 500KB average = 1.25PB in S3; indexed by CDN
- **Indexing overhead:** Elasticsearch inverted index ~30% of raw data = ~300GB index across 50 shards

Order Volume

- **100K orders/day peak (~1.2 orders/sec average)**
- **Peak burst (Cyber Monday):** 500K orders/day (~6 orders/sec)
- **Order storage:** 100 orders/day × 365 × 10 years = 365M orders; ~5KB each = ~1.8TB
- **Fulfillment queue:** Kafka topic; 100K events/day; 7-day retention = ~25GB

Search QPS

- **500K queries/sec globally**
- **Elasticsearch latency target:** <100ms p99
- **Shards needed:** ~500K QPS ÷ 1000 QPS/shard = 500 shards; replicated 3x = 1,500 shard copies
- **Replication factor:** RF=3 for high availability

Bandwidth & Storage

- **Product images:** 5 images/product × 500KB × 500M = 1.25PB; most served from CloudFront CDN
- **Product detail pages:** 300M MAU × 10 page views/month ÷ 30 days = ~100M requests/day = ~1,157 requests/sec
- **Image bandwidth:** ~1,000 requests/sec × 500KB = 500GB/day = 50MB/sec sustained

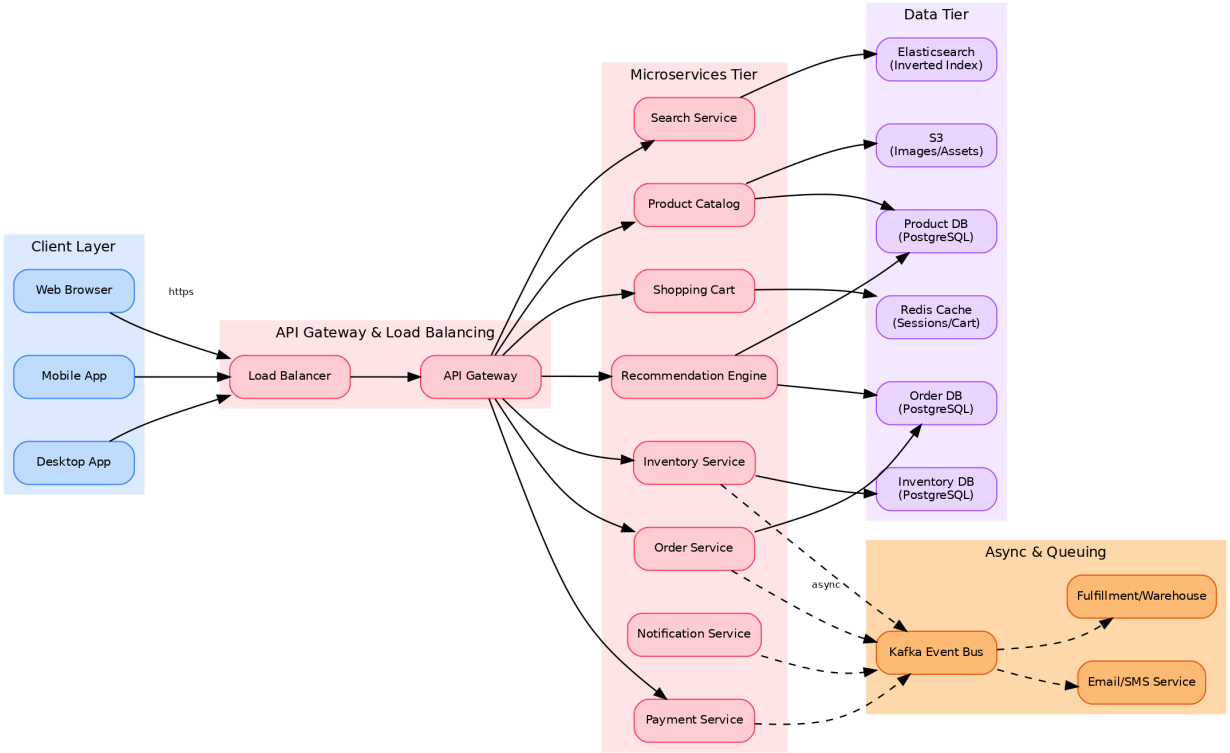
04 System Architecture

Microservices Layered Design

Three-Tier Model

- **Client Layer:** Web browser, mobile app, desktop app; all communicate via HTTPS to load balancer
- **Service Layer:** Stateless microservices (Product Catalog, Search, Inventory, Order, Payment, Cart, Recommendations, Notifications)
- **Data Layer:** Databases (PostgreSQL), caches (Redis), search (Elasticsearch), blob storage (S3)

© Study Guide — For personal use only



Load Balancer & API Gateway

- **Load Balancer:** Distributes traffic across API Gateway instances (e.g., AWS ALB); health checks every 5s
- **API Gateway:** Single entry point; rate limiting, request validation, routing to microservices
- **Service Discovery:** Dynamic service registry (Consul, Eureka, or Kubernetes service mesh); resolves DNS to healthy instances

05 Core Design Decisions

Why This Architecture?

Microservices Decomposition

- **Product Catalog Service:** Read-heavy; immutable metadata; caches product details in Redis
- **Search Service:** Elasticsearch backend; full-text search, faceting, autocomplete
- **Inventory Service:** Single source of truth for stock; enforces constraints via optimistic locking
- **Cart Service:** User's temporary shopping cart; stored in Redis (session-like); TTL = 30 days
- **Order Service:** Creates orders; coordinates with Inventory & Payment; publishes events to Kafka
- **Payment Service:** PCI-compliant; delegates to Stripe/PayPal; handles retries & idempotency
- **Recommendation Service:** Offline batch + online serving; collaborative filtering + content-based
- **Notification Service:** Sends order confirmation, shipment updates, promotional emails

Eventual Consistency in Inventory

 © Study Guide — For personal use only

- **Approach:** Optimistic locking with version numbers; when reservation fails, client retries with updated inventory state
- **Conflict resolution:** 'Sorry, only 2 left' message to user; offer similar items from recommendations
- **Saga pattern:** If payment succeeds but inventory reserve fails, compensation sets payment status to 'refund'
- **TTL on reservations:** Inventory hold expires after 15 minutes if order not confirmed; frees stock for other buyers

Asynchronous Event-Driven Pipeline

- **Kafka broker:** Central event bus; microservices publish domain events (OrderCreated, PaymentConfirmed, InventoryReserved)
- **Event consumers:** Notification Service, Fulfillment Service, Analytics pipeline subscribe to events
- **Benefits:** Loose coupling; if Notification Service down, order still created and Kafka retains event for replay
- **Retention:** 7 days; allows late subscribers to catch up

06 Data Models

Schema Design

Key Tables

```
-- Products Table (Product Catalog Service)
CREATE TABLE products (
product_id BIGINT PRIMARY KEY,
title VARCHAR(255),
description TEXT,
price DECIMAL(10,2),
category_id INT,
seller_id BIGINT,
rating FLOAT,
reviews_count INT,
created_at TIMESTAMP,
updated_at TIMESTAMP,
INDEX idx_category(category_id)
);
-- Inventory Table (Inventory Service)
CREATE TABLE inventory (
sku VARCHAR(50) PRIMARY KEY,
product_id BIGINT NOT NULL,
```


© Study Guide — For personal use only

```
quantity INT,
reserved INT DEFAULT 0,
version INT DEFAULT 0, -- optimistic lock
warehouse_id INT,
updated_at TIMESTAMP,
UNIQUE(product_id, warehouse_id),
INDEX idx_product(product_id)
);

-- Orders Table (Order Service)
CREATE TABLE orders (
order_id BIGINT PRIMARY KEY,
user_id BIGINT NOT NULL,
order_date TIMESTAMP,
total_amount DECIMAL(10,2),
status ENUM('created','payment_pending','confirmed','shipped','delivered'),
shipping_address TEXT,
INDEX idx_user(user_id),
INDEX idx_status(status)
);

-- Order Items (nested in Order Service)
CREATE TABLE order_items (
order_item_id BIGINT PRIMARY KEY,
order_id BIGINT NOT NULL,
product_id BIGINT,
sku VARCHAR(50),
quantity INT,
price DECIMAL(10,2),
INDEX idx_order(order_id)
);
```

PostgreSQL schema excerpt

Redis Keys (Cache & Sessions)

```
-- Cart (Shopping Cart Service)
cart:<user_id> -> { product_id: quantity, ... } // TTL=30d

-- Session
session:<session_id> -> { user_id, logged_in_at } // TTL=24h

-- Product Cache (Product Catalog Service)
product:<product_id> -> JSON(title, price, rating, ...) // TTL=1h
```


© Study Guide — For personal use only

```
-- Inventory Cache
inventory:<sku> -> { quantity, reserved, version } // TTL=5m
```

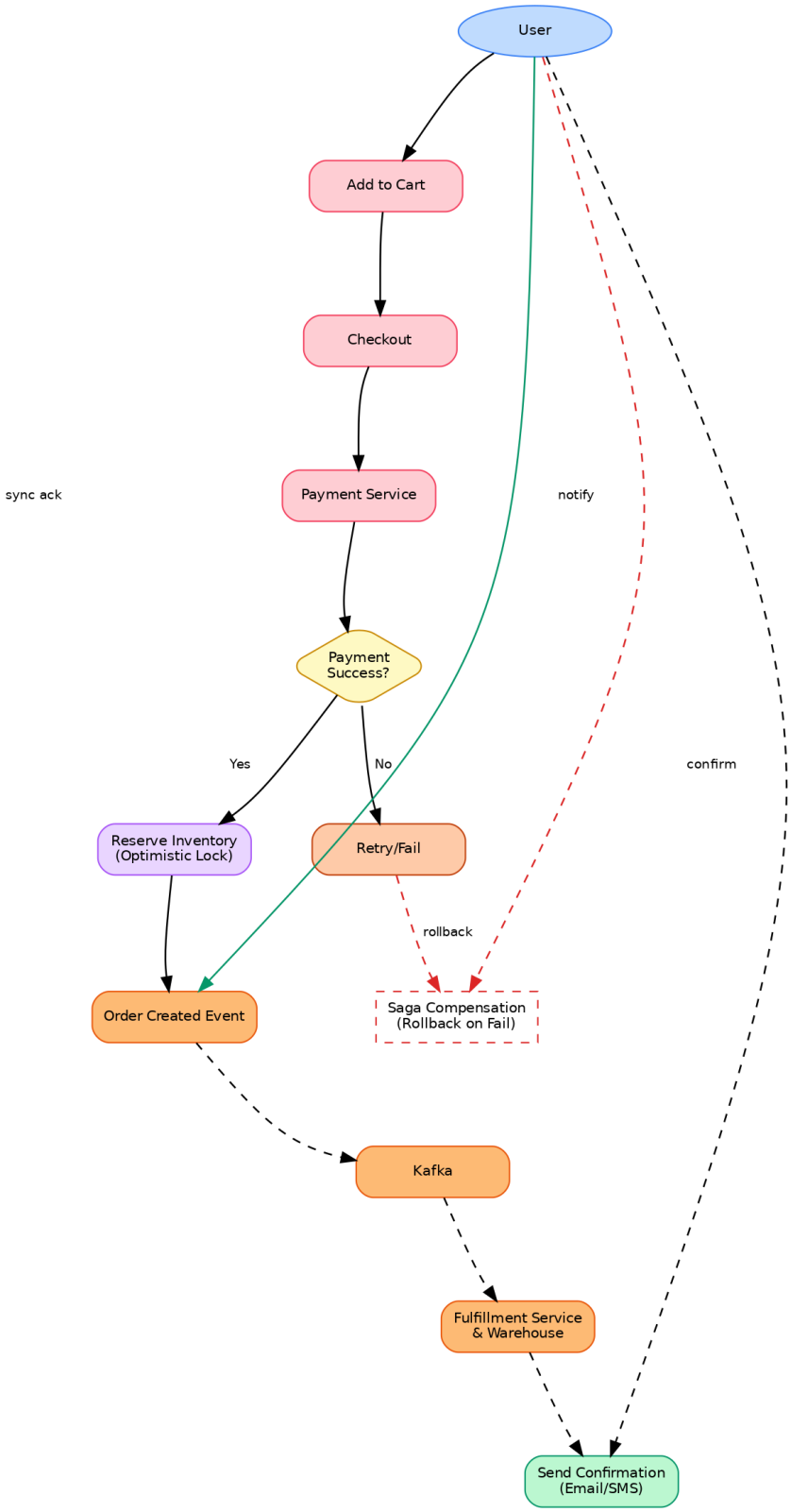
Redis key patterns

07 Order Placement Pipeline

Write Path & Saga Pattern

Request Flow

© Study Guide — For personal use only



1. User adds items to cart (Cart Service stores in Redis)
2. User clicks 'Place Order'; Order Service receives checkout request
3. Order Service initiates payment via Payment Service (calls Stripe API)
4. Payment Service returns success/failure; Order Service checks response

 © Study Guide — For personal use only

5. On success: Order Service calls Inventory Service to reserve stock (with user_id, sku, quantity, version)
6. Inventory Service validates stock and updates with optimistic locking; returns reservation_id
7. Order Service creates Order record (status='confirmed'); publishes OrderCreated event to Kafka
8. Order returns confirmation to client (sync response)
9. Kafka triggers Fulfillment Service (async); updates warehouse system and initiates picking
10. Kafka triggers Notification Service; sends order confirmation email to user

Saga Pattern for Compensation

- **Scenario:** Payment succeeds, but inventory reserve fails (stock exhausted by concurrent order)
- **Compensation:** Order Service marks order as 'failed'; publishes PaymentRefund event; Payment Service processes refund
- **Idempotency:** All operations are idempotent (same request twice = same result); payment refund is no-op if already done
- **Timeouts:** Payment timeout = 30s; inventory reserve timeout = 10s; if exceeded, compensation triggered

Reservation TTL

- **When user adds to cart:** No reservation yet; just a note in Redis cart
- **When checkout begins:** Inventory Service reserves stock for 15 minutes
- **If order confirmed:** Reservation converted to 'sold' (permanent deduction)
- **If order fails or user abandons:** Reservation expires after 15 minutes; stock released back to available
- **Benefit:** Prevents overselling; still allows concurrent checkouts during the 15-minute window

08 Inventory & Concurrency

Preventing Overselling

Optimistic Locking Strategy

- **Version column:** Each inventory row has a version number (starts at 0)
- **Reserve operation:** 'UPDATE inventory SET quantity = quantity - qty, reserved = reserved + qty, version = version + 1 WHERE sku = ? AND version = ?'
- **Conflict:** If version doesn't match, update fails; client retries with fresh inventory state
- **No locks:** Multiple concurrent requests can attempt reserve simultaneously; one succeeds (increments version), others fail and retry
- **Performance:** No blocking; high throughput; occasional retries on contention

Overselling Prevention

© Study Guide — For personal use only

- **Invariant:** quantity ≥ reserved (available = quantity - reserved)
- **Reserve checks:** available ≥ order_qty before attempting update
- **Double-book handling:** If reserve fails, suggest alternative products from Recommendation Service
- **Flash sale scenario:** 100 units available, 1000 concurrent checkout requests; first 100 succeed, rest get 'out of stock'

Reservation TTL Mechanism

- **Background job (every minute):** Scan inventory table for reservations older than 15 minutes
- **Expiry logic:** 'UPDATE inventory SET reserved = reserved - qty WHERE created_at < NOW() - INTERVAL 15min'
- **Notification:** Async job publishes ReservationExpired event (in case Notification Service needs to alert user)
- **Idempotency:** Job is idempotent; if already run twice, second run finds no expired reservations

09 Search Architecture

Elasticsearch at Scale

Indexing Strategy

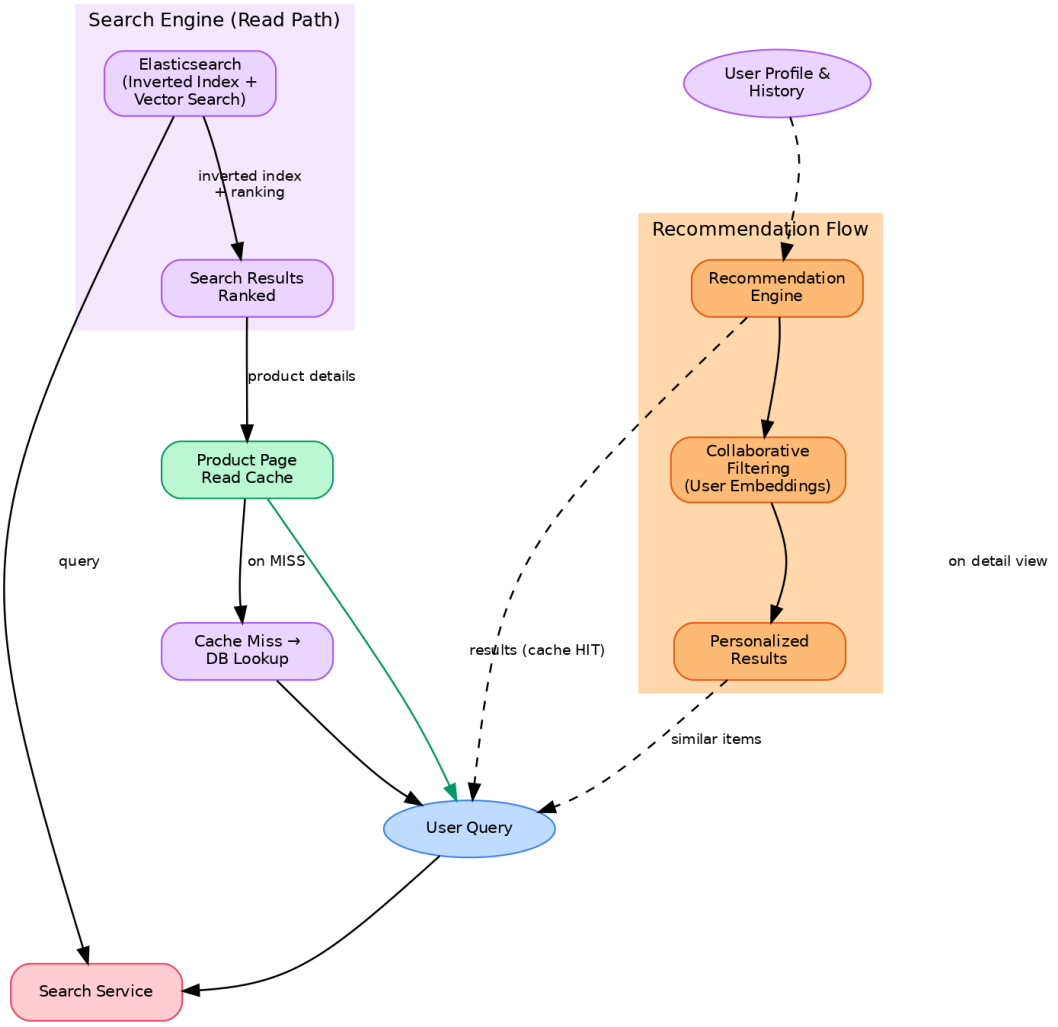
- **Index structure:** product_index (shard by product_id hash mod 50); replica factor = 3
- **Inverted index:** Maps terms (words) to document IDs; e.g., 'samsung' → [prod_123, prod_456, ...]
- **Vector search:** Product embeddings stored as dense_vector field; supports semantic search (e.g., 'cozy reading light')
- **Refresh interval:** 30 seconds (near real-time); new products indexed within 1 minute

Query Features

- **Full-text:** Boolean AND/OR, phrase queries, wildcards, fuzzy match (typo tolerance)
- **Faceting:** Aggregations on category, price_range, rating, seller; enables filter UI
- **Autocomplete:** Prefix query on title field; returns top 10 suggestions in <50ms
- **Ranking:** Relevance score combines TF-IDF + custom signals (rating, reviews_count, sales_velocity)

Read Path with Caching

© Study Guide — For personal use only



- **Search latency target:** <100ms p99 for Elasticsearch + result assembly
- **Cache layer:** Popular searches cached in Redis; e.g., 'iphone 15' → cached results for 1 hour
- **Product detail:** On search result click, fetch product from Redis (HIT expected); on MISS, query PostgreSQL + refresh cache
- **Hit rate:** ~80% cache hits on searches (long-tail of queries isn't repeated); ~95% on product detail (hot products checked repeatedly)

10 Recommendations

Personalization at Scale

Two-Stage Approach

- **Offline (daily batch):** Train collaborative filtering model on user-product interactions; generate top-100 recommendations per user
- **Online (request time):** Fetch cached recommendations; apply real-time filters (remove already-purchased, out-of-stock); return top-20

 © Study Guide — For personal use only

Offline Batch Job

- **Input:** User-product interaction matrix (views, purchases, ratings) from Cassandra or Kafka events
- **Algorithm:** Alternating Least Squares (ALS) matrix factorization or deep neural network embeddings (Word2Vec-like)
- **Output:** Embedding vectors for each user and product; stored in Redis or Cassandra for <10ms lookup
- **Latency:** Nightly job; takes 2–4 hours on Spark cluster; results deployed by morning

Online Serving

- **Request:** User_id → Recommendation Service → fetch embeddings from cache → compute similarity to hot products
- **Filtering:** Remove products in user's purchase history; remove out-of-stock items
- **Personalization:** Rank by predicted rating using additional features (user's price sensitivity, category preferences)
- **A/B testing:** Homepage variants tested on 10% of users; winner (higher CTR/revenue) rolled out to 100%

11 Failure Modes

Scenarios & Recovery

Failure Scenario	Cause	Impact	Mitigation
Payment Service Down	Stripe API timeout; network issue	Checkout blocked; users cannot place orders	Retry with exponential backoff; circuit breaker after 10 failures; queue orders for async processing later
Inventory Service Partition	Database network split	Inventory reads fail; product page shows 'out of stock' incorrectly	Read from cache (stale data ok); eventual consistency; alert ops to repair partition
Search (Elasticsearch) Unavailable	Cluster failure; node crash	Search results unavailable; 'Try again later' message; browse by category as fallback	Replica shard failover (RF=3); add nodes manually; data reindex from backup
Order Service Crashes	Unhandled exception in checkout flow	Orders in-flight lost; user gets 500 error; may retry with duplicate payment	Idempotent request keys (API ensures at-most-once); reconciliation job detects & merges duplicates
Kafka Broker Down	Hardware failure; network issue	Events cannot be published; async pipelines stall; fulfillment delayed	Kafka replication (RF=3); partition leader election; manual failover to replica broker

© Study Guide — For personal use only

Failure Scenario	Cause	Impact	Mitigation
Redis Cache Crash	Memory exhaustion; hardware failure	Cart data lost; users need to re-add items; session expires	Replicate Redis (Redis Sentinel); persistent RDB snapshots; warm cache from database on restart

12 Interview Playbook

How to Ace This Design (45 Minutes)

Opening (5 min): Clarifying Questions

- Confirm scope: Single-vendor Amazon or multi-vendor marketplace? (Single vendor for now; seller model in Phase 2)
- Scale: 1B users, 500M products, 100K orders/day at peak
- Consistency: Real-time inventory or eventual ok? (Real-time for high-velocity; eventual for slow movers)
- Geographic: Single region or global? (Global; VAT & local fulfillment complicate design)

Capacity Estimation (5 min)

- 100K orders/day = ~1.2 orders/sec average; 500K/day peak on Cyber Monday (6 ops/sec)
- 500M products × 2KB = 1TB catalog; 500K QPS search requires ~500 Elasticsearch shards
- Bandwidth: 100M page views/day × 5 images × 500KB = 50MB/sec sustained to CDN

Architecture (10 min): Describe 3 Tiers + Diagrams

- **Client:** Web, mobile, desktop → HTTPS to load balancer
- **Services:** Product Catalog, Search (Elasticsearch), Inventory, Cart (Redis), Order, Payment, Recommendations, Notifications
- **Data:** PostgreSQL (products, orders, users); Redis (cache, sessions, cart); Elasticsearch (search); S3 (images); Kafka (events)
- Draw the diagram with dashed lines for async (Kafka) flows

Deep Dives (20 min): Pick 2–3 Topics

- **Option 1 (Inventory & Concurrency):** Optimistic locking, version columns, reservation TTL, overselling prevention
- **Option 2 (Order Pipeline & Saga):** Transactional flow, compensation on payment failure, idempotency keys
- **Option 3 (Search):** Elasticsearch sharding, inverted index, faceting, autocomplete, caching strategy
- **Option 4 (Recommendations):** Offline ALS training, online embedding lookup, filtering, A/B testing
- **Option 5 (Failure Recovery):** Circuit breakers, fallbacks, replication, Kafka durability

© Study Guide — For personal use only

- **Common follow-ups:** How do you handle payment failure? What if inventory reserve fails after payment succeeds? How do you scale search to 500M products?

Wrap-Up (5 min): Trade-offs

- Eventual consistency vs. strong consistency: Chose eventual for inventory (reservation TTL) to avoid global locks
- Real-time recommendations vs. offline batch: Offline batch sufficient (daily retraining); online serving adds personalization
- Separate microservices vs. monolith: Microservices allow independent scaling (Search vs. Order have different loads)
- End with: 'Key insight is inventory management—preventing overselling at 100K orders/day requires clever concurrency control'

Key Numbers to Memorize

- 1B users, 500M products, 100K orders/day (1.2 ops/sec avg); 500K orders/day peak (6 ops/sec)
- 500K QPS search; Elasticsearch: 500 shards × RF=3; latency <100ms p99
- Inventory: optimistic locking, 15-min reservation TTL, overselling prevention via version checks
- Order pipeline: 4-step (checkout → payment → inventory reserve → order created); saga compensation on failure
- Search: inverted index + vector embeddings; caching strategy ~80% hit rate on searches
- Recommendations: offline ALS batch daily + online serving from cache; <10ms lookup

What Interviewers Are Testing

- **Inventory concurrency:** Can you prevent overselling? Do you understand optimistic locking?
- **Order atomicity:** How do you ensure payment + inventory deduction are transactional? What if one fails?
- **Scale intuition:** 500M products → how many shards? 500K QPS → how many Elasticsearch nodes?
- **Trade-off reasoning:** Why eventual consistency for inventory? Why offline batch for recommendations?
- **Resilience:** What happens if Payment Service times out? If Kafka broker crashes?

 © Study Guide — For personal use only

Key Takeaways

N INVENTORY MANAGEMENT

The hardest part of e-commerce design is preventing overselling while maintaining high throughput. Optimistic locking with version numbers + reservation TTLs elegantly solves this without global locks.

✓ SAGA PATTERN

Distributed transactions are hard. The saga pattern uses events (Kafka) to coordinate compensation steps (refunds, inventory release) when a step fails. Always ensure operations are idempotent.

■ RECOMMENDATION SYSTEMS

Two-stage approach (offline ALS + online serving) balances quality and latency. Batch retraining daily is sufficient; A/B testing validates impact on user engagement.

Further Reading

- **Designing Data-Intensive Applications** (Martin Kleppmann): Chapters on replication, consensus, transactions
- **The Art of Scalability** (Martin Abbott): Multi-tier architecture patterns
- **Release It!** (Michael Nygard): Circuit breaker pattern; resilience strategies
- **Elasticsearch documentation:** Sharding, replication, aggregations
- **AWS WhitePapers:** 'Architecting for Scale on AWS'; 'Multi-Tier Architecture Patterns'

© Study Guide — For personal use only